

Databázy a vizualizácia

SQL – Spúšťače

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

SQL - TRIGGERS

Prečo používame trigger

Trigger je databázový objekt, ktorý sa spustí automaticky pri zmene údajov.

Spúšťa sa pri operáciách:

- INSERT,
- UPDATE,
- DELETE.

Trigger sa nevolá manuálne, vykoná sa automaticky pri zmene v tabuľke.

Používa sa napríklad na:

- kontrolu údajov,
- automatické doplnenie hodnôt,
- audit zmien.

SQL - TRIGGERS

Kedy sa trigger spustí

Trigger sa spúšťa v závislosti od udalosti a času.

Možnosti:

- BEFORE INSERT, AFTER INSERT,
- BEFORE UPDATE, AFTER UPDATE,
- BEFORE DELETE, AFTER DELETE.

BEFORE

- vykoná sa pred zmenou údajov

AFTER

- vykoná sa po zmene údajov

SQL - TRIGGERS

Základná syntax

```
CREATE TRIGGER nazov_triggera  
BEFORE INSERT ON tabulka  
FOR EACH ROW  
BEGIN  
    — telo triggera  
END;
```

FOR EACH ROW znamená, že trigger sa vykoná pre každý riadok zvlášť.

Pri definovaní je potrebné použiť delimiter podobne ako v prípade funkcií a procedúr.

SQL - TRIGGERS

Problém pri UPDATE

Chceme automaticky nastaviť dátum pri zmene riadku.

```
CREATE TRIGGER aktualizuj_datum  
AFTER UPDATE ON t1  
FOR EACH ROW  
BEGIN  
    UPDATE t1  
    SET datetime = NOW()  
    WHERE id = 1;  
END;
```

Tento prístup je zlý. Prečo?

SQL - TRIGGERS

NEW a OLD

V triggeroch máme prístup k hodnotám riadkov.

NEW

- nová hodnota

OLD

- pôvodná hodnota

Dostupnosť:

- INSERT → iba NEW
- DELETE → iba OLD
- UPDATE → OLD aj NEW

SQL - TRIGGERS

Riešenie pomocou NEW

Správny prístup je upraviť hodnotu priamo.

```
CREATE TRIGGER aktualizuj_datum  
BEFORE UPDATE ON t1  
FOR EACH ROW  
BEGIN  
    SET NEW.datetime = NOW();  
END;
```

Nevzniká ďalší UPDATE, teda nevznikne ani rekurzia.

SQL - TRIGGERS

Príklad – audit zmien

Máme tabuľku `blog` a chceme zaznamenávať zmeny do tabuľky `audit`. Po vložení údajov do tabuľky `blog` sa vykoná spúšťač `blog_after_insert`. Pre každý záznam zapíše práve jeden záznam do tabuľky `audit`. Záznam bude obsahovať `id` vkladanych (nových) záznamov do tabuľky `blog` a text `NEW`.

```
CREATE TRIGGER blog_after_insert
AFTER INSERT ON blog
FOR EACH ROW
BEGIN
    INSERT INTO audit(blog_id, changetype)
    VALUES (NEW.id, 'NEW');
END$$
```


SQL - TRIGGERS

Príklad – audit zmien

Druhý spúšťač sa vykoná po zmene údajov v tabuľke blog. V závislosti od stavu stĺpca deleted sa buď zapíše DELETE alebo EDIT.

```
CREATE TRIGGER blog_after_update
AFTER UPDATE ON blog
FOR EACH ROW
BEGIN
    IF NEW.deleted THEN
        INSERT INTO audit(blog_id, changetype)
        VALUES (NEW.id, 'DELETE');
    ELSE
        INSERT INTO audit(blog_id, changetype)
        VALUES (NEW.id, 'EDIT');
    END IF;
END;
```

SQL - TRIGGERS

Použitie triggera

```
INSERT INTO blog(title, content)  
VALUES ('Article One', 'Initial text');
```

```
UPDATE blog  
SET content = 'Edited text'  
WHERE id = 1;
```

```
UPDATE blog  
SET deleted = 1  
WHERE id = 1;
```

Po vykonaní INSERT sa do tabuľky audit zapíše záznam s typom NEW.

Pri prvom UPDATE sa zapíše záznam s typom EDIT, pretože sa mení obsah článku. Pri druhom UPDATE sa nastaví príznak deleted, a preto sa do tabuľky audit uloží záznam s typom DELETE.

SQL - TRIGGERS

Odstránenie triggera

Trigger môžeme odstrániť pomocou príkazu `DROP TRIGGER`.

```
DROP TRIGGER nazov_triggera;
```

Lepšia verzia:

```
DROP TRIGGER IF EXISTS nazov_triggera;
```

Odstránenie je užitočné napríklad pri úprave alebo testovaní triggerov.

SQL - TRIGGERS

Validácia údajov

Trigger môžeme použiť na kontrolu údajov.

```
CREATE TRIGGER kontrola_nazvu  
BEFORE INSERT ON filmy  
FOR EACH ROW  
BEGIN  
    IF CHAR_LENGTH(NEW.nazov) < 3 THEN  
        SIGNAL SQLSTATE '12345'  
        SET MESSAGE_TEXT = 'Nazov je prilis kratky';  
    END IF;  
END;
```

INSERT sa nevykoná ak jeden z nových záznamov obsahuje názov kratší ako 3 znaky. Databáza vráti chybové hlásenie uložené v MESSAGE_TEXT.

- trigger sa viaže na jednu tabuľku,
- vykonáva sa automaticky,
- môže spomaliť operácie,
- treba si dať pozor na nekonečné slučky.